Informe 1: Proceso de Análisis y Diseño de la Solución

Presentado por:

Santiago Ortiz Vargas
Alejandro Serna

Medellín, Antioquia

9 de octubre de 2026

El desafío 2 consiste en el desarrollo de un programa por consola llamada Ude@Book, la cual es una red social que busca posicionarse como una compañía líder entre una amplia comunidad virtual.

Ude@Book requiere una serie de funcionalidades que se va a requerir un buen planteamiento para una buena solución, la cual se evidencia en el actual informe.

Desarrollo de la solución.

- **Administración y formato de archivos en el almacenamiento permanente:**

  Para el almacenamiento de los datos de todos los usuarios se usará archivos de texto plano (.txt), el cual se contará un formato como el siguiente:

  configuracion.txt :

  fecha del dia
  /users/(sub-ruta-X-fija)
  número de usuarios

  usuarios.txt :

  nombre de usuario 1, contraseña 1, fecha inscripción 1, país 1, ciudad 1
  nombre de usuario 2, contraseña 2, fecha inscripción 2, país 2, ciudad 2
  nombre de usuario 3, contraseña 3, fecha inscripción 3, país 3, ciudad 3
  …

  amigos.txt :
  corresponde a una matriz de adyacencia simétrica la cual está optimizada para no repetir valores innecesarios

  [1][10][101][1011][10111][100011][1000011][11000110][...]

  solicitudes.txt :

  índice usuario 1, índice usuario 34, índice usuario 2, índice usuario 45.
  índice usuario 67, índice usuario 34.
  índice usuario 10, índice usuario 8, índice usuario 25.
  …

  mensajes.txt :

  -[fecha 1, L, índice usuario remitente 1, mensaje], [fecha 4, NL, índice usuario remitente 3, mensaje], [fecha 5, L, índice usuario remitente 67, mensaje],
  -[fecha 34, L, índice usuario remitente 5, mensaje], [fecha 47, NL, índice usuario remitente 9, mensaje].

-[fecha 87, índice usuario remitente 2, mensaje].
albumes_usuario.txt :

índice nombre1, índice nombre 4, índice nombre5.
índice nombre 3, índice nombre 7.
índice nombre 7, índice nombre 8, índice nombre10.
…

albumes.txt :

privado
    nombre foto1, #likes, fecha, tamano
    nombre foto2, #likes, fecha, tamano
    nombre foto3, #likes, fecha, tamano
público
    nombre foto4, #likes, fecha, tamano
amigo
    nombre foto5, #likes, fecha, tamano
    nombre foto6, #likes, fecha, tamano
    nombre foto7, #likes, fecha, tamano
público
    nombre foto8, #likes, fecha, tamano
    nombre foto9, #likes, fecha, tamano
    nombre foto10, #likes, fecha, tamano
…

Este formato busca optimizar lo máximo posible de cara a las funcionalidades planteadas, y próximas funcionalidades a futuro a la posible escalabilidad que pueda tener el programa mediante el uso de índices.

Por otro lado, debido a requerimientos de eficiencia se busca evitar manipular lo mínimo posible los archivos de texto plano directamente en el almacenamiento permanente, así que se buscará actualizar la información solamente cuando sea necesario. También se ignorará parcialmente el alto consumo de memoria RAM debido a la carga de información en el transcurso de ejecución del programa.

- **Amigos en común y confirmación de amistad:**

Para la administración de amigos se utilizará una matriz de adyacencia, la cual me dice la relación entre usuarios con un 0 o un 1 (0 para no amigos y 1 para amigos) como se ve en este ejemplo con los usuarios A, B, C ,D y E.

	A	B	C	D	E

A	0	1	0	1	1
B	1	0	1	0	1
C	0	1	0	1	0
D	1	0	1	0	0
E	1	1	0	0	0

Como se puede ver es una matriz simétrica por lo que se optimizará evitando guardar información innecesaria mediante un arreglo lineal el cual representará el triángulo inferior de la matriz tal que:

Triángulo Inferior:

	A	B	C	D	E
A	0	0	0	0	0
B	1	0	0	0	0
C	0	1	0	0	0
D	1	0	1	0	0
E	1	1	0	0	0

Arreglo:
[1][01][101][1100]

Esto permite manejar de manera eficiente los amigos, de manera de que ahorrará memoria y será de fácil acceso y modificación al agregar o eliminar un amigo.

Para amigos en común se utilizará el producto de la matriz de amigos consigo misma, tal que:

	A	B	C	D	E
A	3	1	0	1	1
B	1	3	0	2	1
C	2	0	2	0	1
D	0	2	0	2	1
E	1	1	1	1	2

Esta representa la cantidad de amigos en común entre dos usuarios.

Para la implementación de programas solo necesitaremos una fila de esta para el usuario que inicio sesión, por lo que se calculara una fila de este producto nomás en tiempo de ejecución. Por ejemplo si el usuario es E, entonces calculamos nomas:

E	1	1	1	1	2

Este modelo de solución se implementará con ligeras modificaciones para los mejores amigos, sugerencias de amistad y para aceptar solicitudes de amistad.

- Administración de mensajes:

  Mediante el uso de objetos se piensa enviar mensajes, visualizar y buscar mensajes de una manera eficiente aprovechando los índices indicados en la administración de archivos.

- Visualizar perfil amigo:

  Dado un amigo podremos ver sus álbumes públicos y para amigos, interactuar con ellos de tal modo que podemos ver las rutas de las imágenes de forma circular mediante 4 opciones ya definidas en el enunciado del desafío 2, se busca solucionar este problema mediante funciones y utilización de objetos.

Diagrama de Clases

Para un desarrollo temprano del programa se hizo un diagrama de clases que puede estar sujeta a cambios.

